P2P.ORG SSV INTEGRATION SECURITY AUDIT REPORT

March 19, 2026

MixBytes()

TABLE OF CONTENTS

1. INTRODUCTION 3

1.1 Disclaimer 3

1.2 Security Assessment Methodology 3

1.3 Project Overview 7

1.4 Project Dashboard 8

1.5 Summary of findings 11

1.6 Conclusion 13

2.FINDINGS REPORT 14

2.1 Critical 14

2.2 High 14

H-1 `depositEthAndRegisterValidators()` allows unauthorized users to drain the `SsvToken` balance 14

2.3 Medium 15

M-1 Inability to revoke rights given in `setAllowedSelectorsForClient` and `setAllowedSelectorsForOperator` 15

2.4 Low 16

L-1 Redundant event emission in `P2pSsvProxy.fallback` 16

L-2 Redundant parameter in `P2pSsvProxy.registerValidators` function 17

L-3 Repeated assignment of the same value in the loop 18

L-4 Mitigating error risks in setting `SsvToken` exchange rate 19

L-5 `Unchecked` in `for-loops` is the default Solidity behavior since version 0.8.22 20

L-6 Custom `require` errors are supported by Solidity since version 0.8.26 21

L-7 Outdated value in the comment 22

L-8 Duplicate parameter validation in `makeBeaconDepositsAndRegisterValidators` 23

L-9 Suboptimal storage read operations from `s_allowedSsvOperatorIds` 24

L-10 Incomplete Revocation of Allowed SSV Operator Owners 25

L-11 Beacon Rotation Changes Deterministic Proxy Prediction for Existing Fee Distributors 26

L-12 `makeBeaconDepositsAndRegisterValidators` Uses Clone Predictor Instead of Beacon-Aware Resolution 27

L-13 Unused Reference-Proxy API and Stale Legacy References Remain in the Beacon-Based Factory 28

L-14 Centralization Risks of `P2pUpgradeableBeacon` Ownership 30

3. ABOUT MIXBYTES 31

1. INTRODUCTION

1.1 Disclaimer

The audit makes no statements or warranties about utility of the code, safety of the code, suitability of the business model, investment advice, endorsement of the platform or its products, regulatory regime for the business model, or any other statements about fitness of the contracts to purpose, or their bug free status.

1.2 Security Assessment Methodology

A group of auditors are involved in the work on the audit. The security engineers check the provided source code independently of each other in accordance with the methodology described below:

1. Project architecture review:

- Project documentation review.
- General code review.
- Reverse research and study of the project architecture on the source code alone.

Stage goals
- Build an independent view of the project's architecture.
- Identifying logical flaws.

2. Checking the code in accordance with the vulnerabilities checklist:

- Manual code check for vulnerabilities listed on the Contractor's internal checklist. The Contractor's checklist is constantly updated based on the analysis of hacks, research, and audit of the clients' codes.
- Code check with the use of static analyzers (i.e Slither, Mythril, etc).

Stage goal

Eliminate typical vulnerabilities (e.g. reentrancy, gas limit, flash loan attacks etc.).

3. Checking the code for compliance with the desired security model:

- Detailed study of the project documentation.
- Examination of contracts tests.
- Examination of comments in code.
- Comparison of the desired model obtained during the study with the reversed view obtained during the blind audit.
- Exploits PoC development with the use of such programs as Brownie and Hardhat.

Stage goal

Detect inconsistencies with the desired model.

4. Consolidation of the auditors' interim reports into one:

- Cross check: each auditor reviews the reports of the others.
- Discussion of the issues found by the auditors.
- Issuance of an interim audit report.

Stage goals

- Double-check all the found issues to make sure they are relevant and the determined threat level is correct.
- Provide the Client with an interim report.

5. Bug fixing & re-audit:

- The Client either fixes the issues or provides comments on the issues found by the auditors. Feedback from the Customer must be received on every issue/bug so that the Contractor can assign them a status (either "fixed" or "acknowledged").
- Upon completion of the bug fixing, the auditors double-check each fix and assign it a specific status, providing a proof link to the fix.
- A re-audited report is issued.

Stage goals

- Verify the fixed code version with all the recommendations and its statuses.
- Provide the Client with a re-audited report.

6. Final code verification and issuance of a public audit report:

- The Customer deploys the re-audited source code on the mainnet.
- The Contractor verifies the deployed code with the re-audited version and checks them for compliance.
- If the versions of the code match, the Contractor issues a public audit report.

Stage goals

- Conduct the final check of the code deployed on the mainnet.
- Provide the Customer with a public audit report.

Finding Severity breakdown

All vulnerabilities discovered during the audit are classified based on their potential severity and have the following classification:

Severity	Description
Critical	Bugs leading to assets theft, fund access locking, or any other loss of funds.
High	Bugs that can trigger a contract failure. Further recovery is possible only by manual modification of the contract state or replacement.
Medium	Bugs that can break the intended contract logic or expose it to DoS attacks, but do not cause direct loss funds.
Low	Bugs that do not have a significant immediate impact and could be easily fixed.

Based on the feedback received from the Customer regarding the list of findings discovered by the Contractor, they are assigned the following statuses:

Status	Description
Fixed	Recommended fixes have been made to the project code and no longer affect its security.
Acknowledged	The Customer is aware of the finding. Recommendations for the finding are planned to be resolved in the future.

1.3 Project Overview

The audited scope encompasses several smart contracts designed to deposit ETH to validators and integrate with the SSV Network project. The P2pSsvProxyFactory is responsible for deploying the P2pSsvProxy contract, serving as the entry point for creating the initial deposit of validator stakes and their registration in the SSV Network. Meanwhile, the P2pSsvProxy contract is dedicated to interacting with the SSV Network's functions and managing operator clusters.

1.4 Project Dashboard

Project Summary

Title	Description
Client	P2P.org
Project name	SSV Integration
Timeline	November 14 2023 - March 19 2026
Number of Auditors	3

Project Log

Date	Commit Hash	Note
14.11.2023	9dd4728002d9c275e29e8ba38bcf7d90efc7531b	Commit for the audit
27.11.2023	88e3ecae57e70410ef0ea482ba10d8f541aeac59	Commit for the re-audit
19.06.2024	cfd0a114c760bc4d87121b1c38893f2110f0d474	Commit for the diff-audit
28.06.2024	dfc3e024a3f0779642b43d39ca321cbcf0eb8605	Commit for the re-audit #2
12.09.2025	44cf9bed8625f594495e8ec3526f6bf5f68bc515	Commit with updates
03.03.2026	4dc4e96909c5bb708fa1329332717104a4a03ea1	Commit for migration from SSV to ETH diff-audit
10.03.2026	68980759b09156355ebbb23a33222ed5c07b935f	Commit for the re-audit
12.03.2026	330d21bc67f2bea78d056ad6f7725b3fd8745883	Commit for the re-audit #2

Project Scope

The audit covered the following files:

File name	Link
src/access/Ownable2Step.sol	src/access/Ownable2Step.sol
src/access/OwnableBase.sol	src/access/OwnableBase.sol
src/access/Ownable.sol	src/access/Ownable.sol
src/access/OwnableWithOperator.sol	src/access/OwnableWithOperator.sol
src/assetRecovering/AssetRecoverer.sol	src/assetRecovering/AssetRecoverer.sol
src/assetRecovering/OwnableAssetRecoverer.sol	src/assetRecovering/OwnableAssetRecoverer.sol
src/assetRecovering/OwnableTokenRecoverer.sol	src/assetRecovering/OwnableTokenRecoverer.sol
src/assetRecovering/TokenRecoverer.sol	src/assetRecovering/TokenRecoverer.sol
src/p2pSsvProxy/P2pSsvProxy.sol	src/p2pSsvProxy/P2pSsvProxy.sol
src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol	src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol
src/proxy/P2pBeaconProxy.sol	src/src/proxy/P2pBeaconProxy.sol
src/proxy/P2pUpgradeableBeacon.sol	src/proxy/P2pUpgradeableBeacon.sol

Deployments

Commit 88e3ecae57e70410ef0ea482ba10d8f541aeac59 (previous version 27.11.2023)

Contract	Address
P2pSsvProxyFactory	0x10f4ec919e3e692cb79301e58a7055c783630dfc
P2pSsvProxy	0xbd057f7778485da79c40b252bb09cf1f163d2365

Commit dfc3e024a3f0779642b43d39ca321cbcf0eb8605 (previous version 28.06.2024)

Contract	Address
P2pSsvProxyFactory	0xcb924D4BE3Ff04B2d2116fE116138950373111d9
P2pSsvProxy	0xec17A02B2A8b0C291C0DddE2a00Ca24477c17ED5

Commit 44cf9bed8625f594495e8ec3526f6bf5f68bc515 (previous version 12.09.2025)

Contract	Address
P2pSsvProxyFactory	0x5ed861aec31cCB496689FD2E0A1a3F8e8D7B8824
P2pSsvProxy	0xfdA97AA834c7b65B2AE6e358BE8B11cC28F6CA3f

Commit 330d21bc67f2bea78d056ad6f7725b3fd8745883 (latest version)

Contract	Address
P2pSsvProxyFactory	0x7F12A9904bA021AE3ecE106e10f476b7AfBd830C
P2pSsvProxy	0x94873bCe6fb859255dcDa4c6bA1a559D08F95ef1
P2pUpgradeableBeacon	0x0378Bdc9278f6e334cF7Ac345b744858513FbCfC

1.5 Summary of findings

Severity	# of Findings
Critical	0
High	1
Medium	1
Low	14

ID	Name	Severity	Status
H-1	`depositEthAndRegisterValidators()` allows unauthorized users to drain the `SsvToken` balance	High	Fixed
M-1	Inability to revoke rights given in `setAllowedSelectorsForClient` and `setAllowedSelectorsForOperator`	Medium	Fixed
L-1	Redundant event emission in `P2pSsvProxy.fallback`	Low	Fixed
L-2	Redundant parameter in `P2pSsvProxy.registerValidators` function	Low	Fixed
L-3	Repeated assignment of the same value in the loop	Low	Fixed
L-4	Mitigating error risks in setting `SsvToken` exchange rate	Low	Acknowledged
L-5	`Unchecked` in `for-loops` is the default Solidity behavior since version 0.8.22	Low	Fixed
L-6	Custom `require` errors are supported by Solidity since version 0.8.26	Low	Acknowledged
L-7	Outdated value in the comment	Low	Fixed
L-8	Duplicate parameter validation in `makeBeaconDepositsAndRegisterValidators`	Low	Fixed

L-9	Suboptimal storage read operations from `s_allowedSsvOperatorIds`	Low	Fixed
L-10	Incomplete Revocation of Allowed SSV Operator Owners	Low	Fixed
L-11	Beacon Rotation Changes Deterministic Proxy Prediction for Existing Fee Distributors	Low	Fixed
L-12	`makeBeaconDepositsAndRegisterValidators` Uses Clone Predictor Instead of Beacon-Aware Resolution	Low	Fixed
L-13	Unused Reference-Proxy API and Stale Legacy References Remain in the Beacon-Based Factory	Low	Fixed
L-14	Centralization Risks of `P2pUpgradeableBeacon` Ownership	Low	Fixed

1.6 Conclusion

During the audit, 1 high, 1 medium, and 14 low severity findings have been discovered, confirmed, and either fixed or acknowledged by the developers. An acknowledged finding does not impact the overall security of the project.

During the additional diff audit (comparing commit `cfd0a114c760bc4d87121b1c38893f2110f0d474` with base commit `88e3ecae57e70410ef0ea482ba10d8f541aeac59`), the following attack vectors have been checked:

1. Verification of the correctness of adding/removing validators via `bulkRegisterValidator`/`bulkRemoveValidator`:
   - The code review of the `bulkRegisterValidator` and `bulkRemoveValidator` functions confirmed that the logic for adding and removing validators is implemented correctly. The functions include checks to ensure that validators are added and removed as intended. No vulnerabilities or logical errors were identified.
2. Testing the addition of two validators with the same keys:
   - The code analysis to check the handling of validator key uniqueness showed that the code contains mechanisms to prevent the addition of duplicate validators with the same keys. Appropriate error handling is in place. No vulnerabilities were found.
3. Correctness of Access Rights Handling
   - The code review focused on the handling of access rights across various functions and modules. The analysis confirmed that access control mechanisms are correctly implemented, with appropriate checks to ensure that only authorized entities can perform specific actions.

The code review during the diff audit confirms that the examined components and integrations are secure and implemented correctly according to the specified standards. No critical, high or medium severity vulnerabilities were identified during the analysis. Only findings affecting code optimization and readability, but not security, were found.

2.FINDINGS REPORT

2.1 Critical

Not Found

2.2 High

H-1	`depositEthAndRegisterValidators()` allows unauthorized users to drain the `SsvToken` balance
Severity	High
Status	Fixed in 67086f60

Description

This issue has been identified in the src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol#L421C26-L433 function of the `P2pSsvProxyFactory` contract.
This function permits transferring any specified amount of `SsvToken` to the newly created `SsvCluster` in the `SsvNetwork`. The issue arises from the lack of restrictions on the `SsvPayload.tokenAmount` parameter, enabling users to drain the entire `SsvToken` balance by depositing `32 ETH`. Consequently, these tokens can only be recovered through a multi-step process where the owner has to src/p2pSsvProxy/P2pSsvProxy.sol the total amount from the `SsvCluster` and then src/p2pSsvProxy/P2pSsvProxy.sol it from the corresponding `P2pSsvProxy` back to `P2pSsvProxyFactory`. Moreover, the attacker retains control over their deposit and can withdraw it using their specified `withdrawCredentials`.
This vulnerability is classified as `high` severity due to its potential to block deposits from subsequent clients until the owner intervenes.

Recommendation

We recommend limiting the maximum value of `SSVToken` tokens transferred to a reasonable amount to prevent system block due to actions of unprivileged users and to enhance the system's overall security.

2.3 Medium

M-1	Inability to revoke rights given in `setAllowedSelectorsForClient` and `setAllowedSelectorsForOperator`
Severity	Medium
Status	Fixed in b96c5ab8

Description

The issue is found in src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol and src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol functions of `P2pSsvProxyFactory` contract.
These functions currently grant access rights for invoking `SsvNetwork` functions directly by `client` and `operator` through src/p2pSsvProxy/P2pSsvProxy.sol but do not provide a mechanism to revoke these rights.
This shortfall presents a significant security risk, especially in scenarios where excessive permissions are incorrectly assigned by the `owner`. Additionally, the `ssvNetwork` is an upgradeable proxy contract, and the inability to revoke rights in the event of an interface change further increases the vulnerability.
This issue is classified as `medium` due to the risks associated with the irreversibility of incorrectly granted access.

Recommendation

To mitigate this risk, it is recommended to introduce `onlyOwner` functions that enable the revocation of rights for both the `client` and `operator` in invoking specific functions of the `SsvNetwork` through `P2pSsvProxy`.

2.4 Low

L-1	Redundant event emission in `P2pSsvProxy.fallback`
Severity	Low
Status	Fixed in a20c857b

Description

This issue is found in the src/p2pSsvProxy/P2pSsvProxy.sol#L164C56-L164C56 function of the `P2pSsvProxy` contract.
The current implementation emits an event before a `revert` operation. However, it's important to note that a transaction revert fully reverses all state changes made during the transaction, including the event log. This renders the event emission redundant and ineffective, as it will be erased upon the execution of `revert`.

Recommendation

We recommend removing this event emission.

L-2	Redundant parameter in `P2pSsvProxy.registerValidators` function
Severity	Low
Status	Fixed in 901b78e1

Description

This issue is found in the src/p2pSsvProxy/P2pSsvProxy.sol function if the `P2pSsvProxy` contract. This function, which is only accessible from the `factory`, is invoked within the src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol function. Before this invocation, a validation check confirms that the `P2pSsvProxy` has been created with a specific `feeDistributorInstance`. This fee distributor is immutable and is assigned as the src/p2pSsvProxy/P2pSsvProxy.sol storage variable within `P2pSsvProxy`. Given this setup, explicitly passing the `feeDistributor` parameter to `registerValidators` is unnecessary.

Recommendation

We recommend removing the `feeDistributor` parameter from the `registerValidators` function and utilizing the existing `s_feeDistributor` storage variable instead.

L-3	Repeated assignment of the same value in the loop
Severity	Low
Status	Fixed in 20b6a086

Description

A gas efficiency issue is found in the src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol function of the `P2pSsvProxyFactory` contract.
The problem arises because the `withdrawCredentials` variable is repeatedly assigned the same value within a loop. This repetitive assignment is unnecessary and increases gas costs for executing this function.

Recommendation

We recommend moving the declaration of the `withdrawCredentials` variable outside the loop.

L-4	Mitigating error risks in setting `SsvToken` exchange rate
Severity	Low
Status	Acknowledged

Description

This issue is found in the src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol function of the `P2pSsvProxyFactory` contract.
Currently, the contract owner can arbitrarily set the exchange rate of `SsvToken` to `ETH` within given limits. This poses the risk of potential errors by the owner.

Recommendation

We recommend integrating an oracle for the `SsvToken` to derive its market price from real-time market data.

Client's Commentary

> This is a good idea. But we deliberately decided to do it the simple way and accept the risk to ourselves. It's important that the client is not risking here. They see the exchange rate and proceed only if they agree.

L-5	`Unchecked` in `for-loops` is the default Solidity behavior since version 0.8.22
Severity	Low
Status	Fixed in dfc3e024

Description

While the project is using Solidity version 0.8.24, it is redundant to use `unchecked` incrementing of the `for-loop` variables since Solidity implements such behavior by default starting from version 0.8.22.

Recommendation

We recommend using a standard `for-loop` increment notation for better code readability.

Client's commentary

> Fixed in dfc3e024.

L-6	Custom `require` errors are supported by Solidity since version 0.8.26
Severity	Low
Status	Acknowledged

Description

Currently, custom errors are implemented using the `if(condition) revert(CustomError)` flow, which is the only way to implement custom errors in Solidity versions before 0.8.26. However, the latest version of Solidity already supports the `require(bool, CustomError)` flow.

Recommendation

We recommend considering upgrading Solidity to the latest version and replacing the `if - revert` flow with the `require` flow when it is beneficial for code readability.

Client's commentary

> We prefer staying on 0.8.24 to match SSV Solidity version exactly. Switched to evm_version = 'cancun' for the same reason.

L-7	Outdated value in the comment
Severity	Low
Status	Fixed in dfc3e024

Description

While the constant value `MAX_ALLOWED_SSV_OPERATOR_IDS` was updated from 16 to 24, the corresponding comment `limited to 16 IDs` in the code was not updated.

Related code: src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol

Recommendation

We recommend updating the comment to keep it consistent with the code.

Client's commentary

> Fixed in dfc3e024.

L-8	Duplicate parameter validation in `makeBeaconDepositsAndRegisterValidators`
Severity	Low
Status	Fixed in dfc3e024

Description

The values passed to the `makeBeaconDepositsAndRegisterValidators` function are validated twice: within the function itself and in the `P2POrgUnlimitedEthDepositor.makeBeaconDeposit` function. The validation in `makeBeaconDepositsAndRegisterValidators` can be omitted without compromising code security.

Related code: src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol

Recommendation

We recommend considering to remove the redundant validation from `makeBeaconDepositsAndRegisterValidators`.

Client's commentary

> Fixed in dfc3e024.

L-9	Suboptimal storage read operations from `s_allowedSsvOperatorIds`
Severity	Low
Status	Fixed in dfc3e024

Description

The `s_allowedSsvOperatorIds` array is stored as a static-size array with the size of `MAX_ALLOWED_SSV_OPERATOR_IDS`. To obtain data from that array, the current implementation generally reads the whole array from storage, regardless of whether it uses all its capacity or not. This is a suboptimal solution in terms of gas usage.

Related code:

- src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol
- src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol

Recommendation

We recommend developing a more optimal solution to save gas.

Client's commentary

> Fixed in dfc3e024.

L-10	Incomplete Revocation of Allowed SSV Operator Owners
Severity	Low
Status	Fixed in 68980759

Description

`removeAllowedSsvOperatorOwners()` removes an owner from `s_allowedSsvOperatorOwners`, but does not clear `s_allowedSsvOperatorIds[owner]`.

At the same time, `onlyAllowedOperators` and `onlyAllowedOperatorsByOwner` validate operators only via `s_allowedSsvOperatorIds`, without verifying that the owner is still in `s_allowedSsvOperatorOwners`.

As a result, a removed owner may remain effectively usable in validator registration flows (`registerValidators(...)` and `registerValidatorsEth(...)`) until IDs are cleared separately.

This creates an authorization inconsistency: owner removal is not an immediate full revoke. In an operational mistake scenario, validators can still be registered with operators that were intended to be deauthorized.

Given required privileged setup/actions and available manual cleanup, severity is **low**.

Recommendation

We recommend revoking ssv operator roles atomically by clearing `s_allowedSsvOperatorIds[owner]` inside `removeAllowedSsvOperatorOwners()`. Additionally, enforce defense-in-depth in `onlyAllowedOperators*` by also requiring `s_allowedSsvOperatorOwners.contains(owner)`.

L-11	Beacon Rotation Changes Deterministic Proxy Prediction for Existing Fee Distributors
Severity	Low
Status	Fixed in 68980759

Description

The factory supports two deterministic deployment paths for proxies: clone-based and beacon-based.

For beacon deployments, `predictP2pSsvProxyAddressBeacon(...)` includes the current `s_beacon` value in the CREATE2 bytecode hash. Therefore, when `setBeacon(...)` is called with a new beacon address, the predicted proxy address for the same fee distributor changes.

At the same time, deployment flows use `_createP2pSsvProxyAuto(...)`, which follows the current beacon setting. This can cause the system to resolve/deploy a different proxy address for the same fee distributor after beacon updates.

As a result, address prediction can drift across time and no longer point to previously deployed proxy instances. This is mainly an operational consistency issue (indexing/discovery/routing confusion), not a direct loss-of-funds vector.
Given that multiple proxies per fee distributor are acceptable by design, severity is **low**.

Recommendation

We recommend introducing a canonical on-chain mapping from fee distributor to active proxy (or primary proxy), and using it as the first source of truth in all deployment/routing flows.

Alternatively, if minimizing migration and operational branching is a priority, consider standardizing on beacon-only proxy creation going forward (i.e., deprecate clone deterministic creation for new deployments). This removes dual-path address derivation and reduces long-term prediction/routing complexity.

L-12	`makeBeaconDepositsAndRegisterValidators` Uses Clone Predictor Instead of Beacon-Aware Resolution
Severity	Low
Status	Fixed in 68980759

Description

`makeBeaconDepositsAndRegisterValidators(...)` resolves the proxy address using `predictP2pSsvProxyAddress(...)`, which corresponds to the clone prediction path.

However, when beacon mode is active, proxies are deployed at addresses derived by `predictP2pSsvProxyAddressBeacon(...)`.

Because of this mismatch, the function may check code existence at a clone-predicted address and revert with `P2pSsvProxyFactory__P2pSsvProxyDoesNotExist(...)`, even if the correct beacon proxy already exists.

This affects a specific registration flow and can lead to avoidable operational failures/reverts in beacon mode. The issue is limited in scope and does not directly imply fund theft, so severity is **low**.

Recommendation

We recommend making prediction functions mode-aware and explicit (clone vs beacon), and avoid using raw prediction as the only way to discover existing proxies after beacon changes.

Alternatively, to simplify maintenance and avoid repeated migration edge cases, migrate to a beacon-only creation model for new proxies and deprecate clone-based deterministic creation paths. This removes predictor mismatch classes entirely.

L-13	Unused Reference-Proxy API and Stale Legacy References Remain in the Beacon-Based Factory
Severity	Low
Status	Fixed in 330d21bc

Description

The factory contains a legacy reference-proxy path that is no longer used by production deployment logic:

- `s_referenceP2pSsvProxy` at src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol
- `setReferenceP2pSsvProxy(...)` at src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol
- `getReferenceP2pSsvProxy()` at src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol
- `P2pSsvProxyFactory__ReferenceP2pSsvProxySet` / related interface declarations at src/p2pSsvProxyFactory/IP2pSsvProxyFactory.sol, src/p2pSsvProxyFactory/IP2pSsvProxyFactory.sol, and src/p2pSsvProxyFactory/IP2pSsvProxyFactory.sol.

Current proxy creation does not use this path. New proxies are created through the beacon flow only: src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol, src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol, src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol.

In addition, `P2pSsvProxyFactory__InvalidEthValue` at src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol is declared but unused.

There are also stale legacy comments/docs around the same area:

- comments referencing `depositEthAndRegisterValidators` at src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol, src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol, src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol, and src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol, even though that entrypoint is deprecated and always reverts at src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol and src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol
- the beacon comment at src/p2pSsvProxyFactory/P2pSsvProxyFactory.sol still says the legacy clone path is used when beacon is unset, but `_createP2pSsvProxyAuto(...)` now reverts instead

This is only a maintainability issue: the codebase exposes dead configuration/API surface and outdated references that should be removed or updated.

Recommendation

We recommend removing the unused reference-proxy artifacts from the factory and interface:

- `s_referenceP2pSsvProxy`

- `setReferenceP2pSsvProxy(...)`
- `getReferenceP2pSsvProxy()`
- `P2pSsvProxyFactory__ReferenceP2pSsvProxySet`
- `P2pSsvProxyFactory__InvalidEthValue`

Then update the remaining comments and docs to match the current beacon-only flow and the deprecated status of `depositEthAndRegisterValidators`.

L-14	Centralization Risks of `P2pUpgradeableBeacon` Ownership
Severity	Low
Status	Fixed in 330d21bc

Description

`P2pUpgradeableBeacon` is controlled by a single-step owner:

- owner storage at src/proxy/P2pUpgradeableBeacon.sol
- `upgradeTo(...)` at src/proxy/P2pUpgradeableBeacon.sol
- `transferOwnership(...)` at src/proxy/P2pUpgradeableBeacon.sol

This is weaker than the factory ownership model, which already uses `Ownable2Step` via src/access/OwnableWithOperator.sol.

Since all beacon proxies depend on this beacon, the beacon owner can replace the implementation for the entire proxy owned by factory. That makes the beacon owner a critical trusted role. The factory owner is also privileged and should be treated similarly.

Recommendation

We recommend using a 2-step transferring ownership pattern for the beacon, and putting both the beacon owner and factory owner behind a multisig.

Client's commentary

> Both the beacon owner and the factory owner are secured behind a multisig.

3. ABOUT MIXBYTES

MixBytes is a team of blockchain developers, auditors and analysts keen on decentralized systems. We build opensource solutions, smart contracts and blockchain protocols, perform security audits, work on benchmarking and software testing solutions, do research and tech consultancy.

Contacts

https://github.com/mixbytes/audits_public

https://mixbytes.io/

hello@mixbytes.io

https://twitter.com/mixbytes